



Универзитет „Св. Кирил и Методиј“ во Скопје
**ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И
КОМПЈУТЕРСКО ИНЖЕНЕРСТВО**

Континуирана интеграција и испорака

Тема: Докеризација и испорака со Кубернетес на веб апликација за сместувања

Ментор:

проф. др. Панче Рибарски

Изработила:

Ана Коловска, 221226

Contents

Апстракт	3
Вовед.....	4
Главен дел	5
Docker.....	5
CI Pipeline	6
Kubernetes	6
Заклучок.....	9
Линкови	10

Апстракт

Овој проект ја прикажува Dockerизацијата и Kubernetes деплојментот на веб апликацијата **Accommodations App**, која претходно е развиена како проект по друг предмет. Главната цел е постоечката апликација да биде контејнеризирана, автоматизирано изградена и распоредена во Kubernetes околина со примена на современи DevOps практики.

Апликацијата се состои од **React (Vite)** frontend, **Spring Boot** backend и **PostgreSQL** база на податоци. За секоја компонента се изработени Docker имиџи, кои автоматски се поставуваат на Docker Hub преку **GitHub Actions**. Распоредувањето во Kubernetes е реализирано со користење на Deployment, StatefulSet, Service, ConfigMap, Secret, Namespace и Ingress ресурси. Дополнително, имплементиран е **GitOps** пристап со користење на **Argo CD**, кој овозможува автоматска синхронизација помеѓу Git репозиториумот и Kubernetes кластерот.

Како резултат е добиен целосно функционален систем кој овозможува едноставно, сигурно и автоматизирано градење, дистрибуција и распоредување на нови верзии на апликацијата. Проектот демонстрира практична примена на Docker, Kubernetes, GitHub Actions и Argo CD како современи алатки за развој и управување со cloud-native апликации.

Вовед

Во современиот развој на софтвер, контејнеризацијата и оркестрацијата на апликации претставуваат едни од најзначајните технологии за развој, тестирање и продукциско распоредување на софтверски системи. Користењето на Docker овозможува апликациите и сите нивни зависности да бидат спакувани во преносливи контејнери, со што се обезбедува идентично однесување на апликацијата без разлика на околината во која се извршува. Од друга страна, Kubernetes овозможува автоматизирано управување со контејнерите, нивно распоредување, скалирање, обновување и комуникација помеѓу различните компоненти на системот.

Целта на оваа проектна задача е да се изврши Dockerизација и Kubernetes деплојмент на постоечката веб апликација **Accommodations App**, која претходно е развиена како проект по друг предмет. Наместо повторна имплементација на функционалностите, фокусот на овој проект е поставен на современите DevOps практики и начинот на кој една постоечка апликација може да биде подготвена за сигурно и автоматизирано распоредување.

Апликацијата се состои од три главни компоненти: **Spring Boot** backend кој обезбедува REST API, **React (Vite)** frontend кој го имплементира корисничкиот интерфејс и **PostgreSQL** база на податоци во која се чуваат сите информации за сместувањата, домаќините, државите и корисниците. Секоја од овие компоненти е контејнеризирана и распоредена како посебен ресурс во Kubernetes кластер.

Во рамки на проектот се изработени Docker имиџи за backend и frontend апликацијата, кои автоматски се објавуваат на Docker Hub преку **GitHub Actions**. За управување со Kubernetes ресурсите се користат Deployment, StatefulSet, Service, ConfigMap, Secret, Ingress и Namespace манифести. Дополнително, за автоматска синхронизација на Kubernetes кластерот со Git репозиториумот е користен **Argo CD**, со што е имплементиран GitOps пристап за управување со инфраструктурата.

Како резултат на проектот е добиено целосно автоматизирано решение кое овозможува секоја промена во изворниот код да биде изградена, контејнеризирана, поставена на Docker Hub и подготвена за автоматско распоредување во Kubernetes околина. Со тоа значително се намалува потребата од рачни интервенции, се олеснува одржувањето на системот и се обезбедува поголема сигурност и повторливост на процесот на деплојмент.

Главен дел

Docker

Првиот чекор во реализацијата на проектот беше контејнеризацијата на сите компоненти од апликацијата. Апликацијата е составена од три главни дела: React frontend, Spring Boot backend и PostgreSQL база на податоци. За frontend и backend беа изработени посебни Dockerfile датотеки, додека за базата на податоци беше искористена официјалната Docker слика на PostgreSQL.

Backend апликацијата е изградена со Maven, по што се креира Docker имиц кој содржи само потребните датотеки за извршување на Spring Boot апликацијата. На овој начин се добива лесен и пренослив контејнер кој може да се стартува на било која машина што има Docker.

Frontend апликацијата е развиена со React и Vite. Во Dockerfile се користи multi-stage build. Во првата фаза се инсталираат зависностите и се генерира production build на апликацијата, додека во втората фаза се користи Nginx за сервирање на статичките датотеки. Дополнително беше конфигуриран Nginx со цел правилно функционирање на React Router при директен пристап до различните рути на апликацијата.

За базата на податоци се користи официјалната PostgreSQL Docker слика, при што корисничкото име, лозинката и името на базата се конфигурираат преку environment променливи.

По успешно креирање на Docker имиците, тие автоматски се поставуваат на Docker Hub, од каде Kubernetes може да ги преземе при деплојментот на апликацијата.

```
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /app
COPY . .
RUN chmod +x mvnw
RUN ./mvnw clean package -DskipTests
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

FROM node:22 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ARG VITE_API_URL=http://accommodations.local/api
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

CI Pipeline

За автоматизација на процесот на градење и објавување на апликацијата беше искористен GitHub Actions. Pipeline-от се активира при секој push или pull request кон главната гранка на проектот.

Процесот е поделен на три независни jobs.

Првиот job е задолжен за backend апликацијата. Во рамки на workflow-от автоматски се стартува PostgreSQL сервис, по што се конфигурира Java околината, се преземаат Maven зависностите и се извршуваат build и тестовите на Spring Boot апликацијата.

Вториот job го обработува frontend делот. Се инсталира Node.js, се преземаат сите npm зависности, се извршува lint проверка на кодот и се генерира production build на React апликацијата.

Доколку двата претходни jobs завршат успешно, започнува третиот job. Во него GitHub Actions се најавува на Docker Hub со користење на credentials зачувани како GitHub Secrets, по што автоматски се градат Docker имиците за backend и frontend и се поставуваат на Docker Hub. Секој имиц се објавува со ознаката **latest**, како и со ознака која го содржи SHA идентификаторот на commit-от, што овозможува следење на сите верзии на апликацијата.

Со ова значително се намалува потребата од рачно градење и поставување на Docker имиците, а целиот процес се извршува автоматски по секоја промена во изворниот код.

Kubernetes

По успешната Dockerизација, апликацијата беше распоредена во локален Kubernetes кластер креиран со k3d. За секоја компонента беа изработени Kubernetes манифести кои овозможуваат автоматизирано креирање и управување со ресурсите.

Backend и frontend се распоредени како Deployment ресурси, со што Kubernetes автоматски се грижи за нивното стартување, рестартирање и достапност. PostgreSQL е распоредена како StatefulSet бидејќи базата на податоци има потреба од постојано складирање на податоците и стабилен идентитет на подот.

За комуникација помеѓу компонентите се користат Kubernetes Service ресурси. Backend комуницира со PostgreSQL преку ClusterIP сервис, додека frontend испраќа HTTP барања кон backend преку Ingress.

Конфигурациските параметри, како URL адресите и останатите поставки, се чуваат во ConfigMap ресурси, додека чувствителните информации, како лозинките за базата на податоци, се складираат во Secret ресурси.

За пристап до апликацијата од веб прелистувач е конфигуриран Ingress ресурс кој го насочува сообраќајот кон frontend или backend зависно од URL патеката. На тој начин frontend е достапен преку `http://accommodations.local`, додека REST API-то е достапно преку `http://accommodations.local/api`.

Како дополнување на Kubernetes деплојментот беше имплементиран и GitOps пристап со користење на Argo CD. Argo CD го следи Git репозиториумот во кој се наоѓаат Kubernetes манифестите и автоматски ги синхронизира промените со кластерот. На овој начин целата инфраструктура останува верзионирана во Git, а секоја промена може лесно да се следи, врати или повторно распореди.

```
name: backend
namespace: accommodations
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: happykhat/rentals-backend:latest
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_DATASOURCE_URL
              valueFrom:
                configMapKeyRef:
                  name: backend-config
                  key: SPRING_DATASOURCE_URL
            - name: SPRING_JPA_HIBERNATE_DDL_AUTO
              valueFrom:
                configMapKeyRef:
                  name: backend-config
                  key: SPRING_JPA_HIBERNATE_DDL_AUTO
            - name: SPRING_DATASOURCE_USERNAME
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_USER
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: postgres-secret
                  key: POSTGRES_PASSWORD
```

```
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: accommodations
spec:
  selector:
    app: backend
  ports:
    - port: 8080
      targetPort: 8080
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: accommodations
data:
  SPRING_DATASOURCE_URL: "jdbc:postgresql://postgres:5432/rentals"
  SPRING_JPA_HIBERNATE_DDL_AUTO: "update"
  SPRING_DATASOURCE_DB: "rentals"
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: accommodations-ingress
  namespace: accommodations
  annotations:
    nginx.ingress.kubernetes.io/use-regex: "true"
spec:
  rules:
    - host: accommodations.local
      http:
        paths:
          - path: /api/
            pathType: Prefix
            backend:
              service:
                name: backend
                port:
                  number: 8080
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
```

Заклучок

Во рамки на овој проект беше успешно извршена Dockerизација и Kubernetes деплојмент на веб апликацијата **Accommodations App**. Сите компоненти на системот – React frontend, Spring Boot backend и PostgreSQL базата на податоци – беа успешно контејнеризирани и распоредени во Kubernetes кластер, при што секоја компонента е конфигурирана да комуницира со останатите преку Kubernetes ресурсите.

Дополнително беше воспоставен автоматизиран CI процес со користење на **GitHub Actions**, кој овозможува автоматско градење, тестирање и поставување на Docker имиците на Docker Hub при секоја промена во изворниот код. Со интеграцијата на **Argo CD** беше имплементиран GitOps пристап, при што Kubernetes кластерот автоматски се синхронизира со Git репозиториумот што ги содржи Kubernetes манифестите.

Изработката на проектот овозможи практична примена на современи DevOps технологии и концепти како што се Docker, Kubernetes, GitHub Actions и Argo CD. Во текот на имплементацијата беа стекнати знаења за контејнеризација на апликации, управување со Kubernetes ресурси, автоматизација на процесите за континуирана интеграција и примена на GitOps пристап за континуирано распоредување.

Конечниот резултат е целосно функционална и автоматизирана околина во која апликацијата може лесно да се изгради, распореди и ажурира со минимална рачна интервенција. Ваквиот начин на работа го олеснува одржувањето на системот, ја зголемува сигурноста на процесот на деплојмент и претставува добра основа за понатамошен развој и проширување на апликацијата.

Линкови

GitHub (апликација):

https://github.com/akolovska/Accommodations_app

GitHub (Kubernetes manifests):

<https://github.com/akolovska/Accommodations-app-manifests>

Docker Hub Backend:

<https://hub.docker.com/r/happykhat/rentals-backend>

Docker Hub Frontend:

<https://hub.docker.com/r/happykhat/rentals-frontend>